

**7/20 미팅**

**HeteroMAS: Heterogeneous Agent를 위한 KV-Cache Communication**

**하서준**

DGIST

2026년 7월 20일

LatentMAS · RecursiveMAS · Cache-to-Cache · HeteroMAS

# 이번 발표의 목적

## 비교할 기존 구조

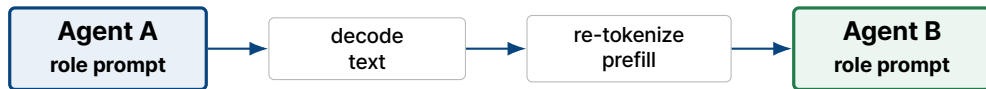
- ▶ **LatentMAS**: training-free latent thought + KV working memory
- ▶ **RecursiveMAS**: inner/outer recursive link 기반 latent MAS
- ▶ **Cache-to-Cache**: pairwise LLM cache fusion

## 우리 구조

- ▶ heterogeneous role-specialized agents
- ▶ hidden state 전달 제거
- ▶ sender KV를 receiver-compatible prefix KV로 번역
- ▶ receiver KV와 fusion하지 않고 prepend



## 문제 설정: Text Communication의 병목



### 정보 손실

고차원 내부 표현을 선형 text token으로 압축한다.

### 지연

중간 agent가 token-by-token decoding을 해야 한다.

### 역할 불일치

agent마다 prompt와 objective가 다르다면 같은 text도 다르게 해석된다.

**핵심 질문:** heterogeneous MAS에서 text 대신 어떤 latent state를 넘길 것인가?

# LatentMAS: latent thought와 KV working memory

## Latent thought generation

$$h_t = f_\theta(e_{\leq t}), \quad e_{t+1} = h_t W_a$$

$$W_a \approx W_{\text{out}}^\dagger W_{\text{in}}$$

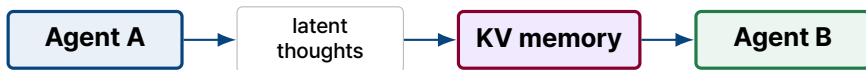
token을 decode하지 않고 last-layer hidden state를 다음 input embedding으로 다시 넣는다.

## KV working memory transfer

$$M_A = \{(K_A^\ell, V_A^\ell)\}_{\ell=1}^L$$

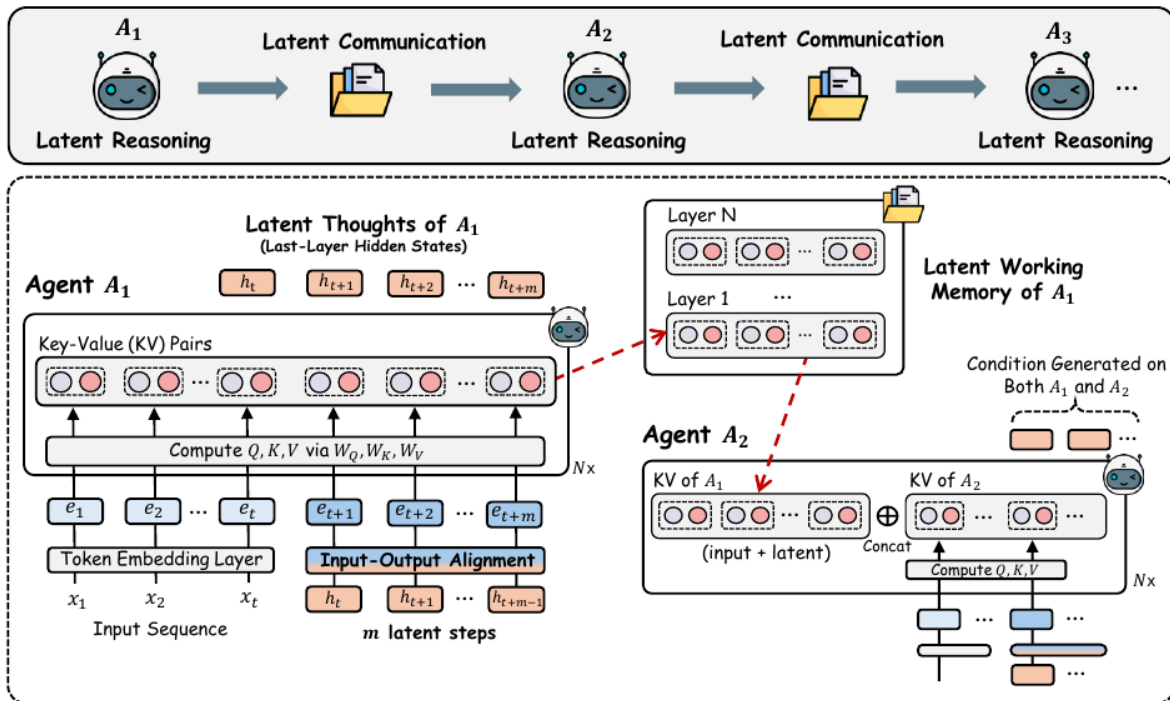
$$K_B^\ell \leftarrow [K_A^\ell; K_B^\ell], \quad V_B^\ell \leftarrow [V_A^\ell; V_B^\ell]$$

agent A가 만든 layer-wise KV cache를 agent B의 prefix memory 처럼 붙인다.



# LatentMAS overview

## Latent Collaboration (LatentMAS)



# LatentMAS의 한계: heterogeneous setting

## 논문 구조의 강한 가정

- ▶ 같은 모델 family 또는 같은 shape의 layer-wise KV를 전제로 하기 쉽다.
- ▶ 서로 다른 layer 수, KV head 수, head dimension에 대한 변환은 main method가 아니다.
- ▶ heterogeneous extension은 adapter가 필요하다는 future direction에 가깝다.



따라서 HeteroMAS에서는 “KV를 넘긴다”가 아니라, sender KV를 receiver가 읽을 수 있는 prefix KV로 번역해야 한다.

# RecursiveMAS: inner-loop와 outer communication

## Inner recursive link

$$R_{in}(h) = h + W_2 \sigma(W_1 h)$$

$$\mathcal{L}_{in} = 1 - \cos(R_{in}(H), \text{Emb}(y))$$

latent state를 정답 label 또는 teacher-generated target 쪽으로 정렬한다.

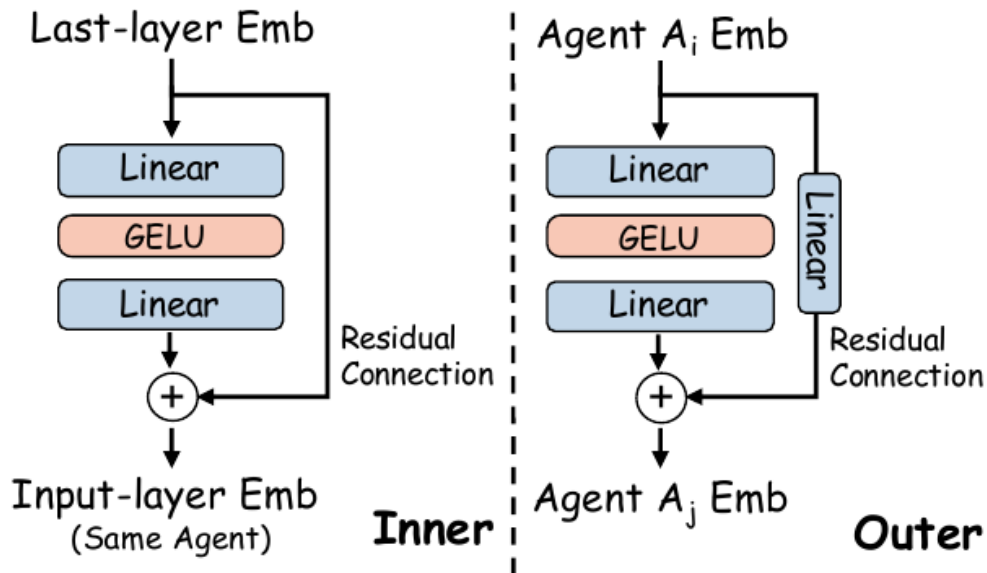
## Outer communication

$$R_{out}(h) = W_3 h + W_2 \sigma(W_1 h)$$

agent 사이에는 주로 last-layer hidden representation을 adapter로 변환해 전달한다.

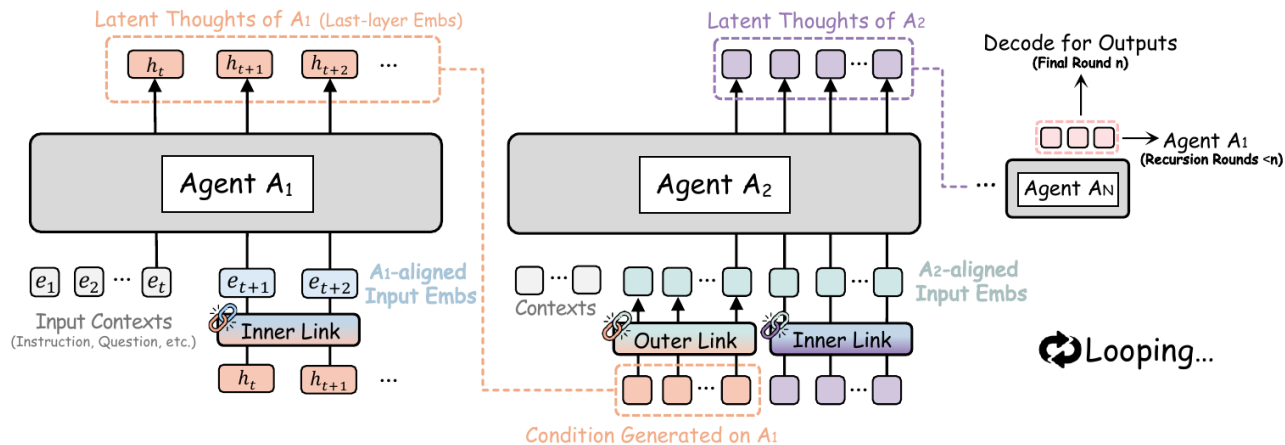


## RecursiveLink





# RecursiveMAS overview



# RecursiveMAS의 약점

## 학습 supervision 문제

- ▶ closed-form QA에서는 정답 label이 명확하다.
- ▶ planner/refiner target은 teacher rewrite에 의존한다.
- ▶ open-ended MAS에서는 role-level gold trace가 애매하다.

## 전달 단위 문제

- ▶ layer-wise KV가 아니라 hidden representation 전달이다.
- ▶ transformer attention memory 자체를 넘기지 않는다.
- ▶ heterogeneous agent 간 shape mismatch를 별도 처리해야 한다.

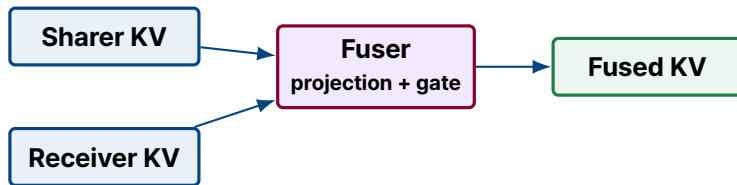
우리 실험에서는 RecursiveMAS inner-loop는 비교 공정성을 위해 유지하고, cross-agent communication만 KV prefix translation으로 교체한다.

# Cache-to-Cache: pairwise cache fusion

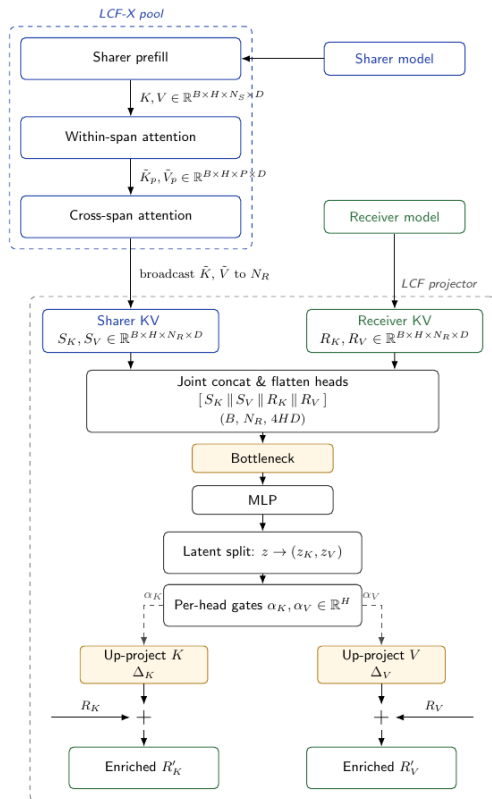
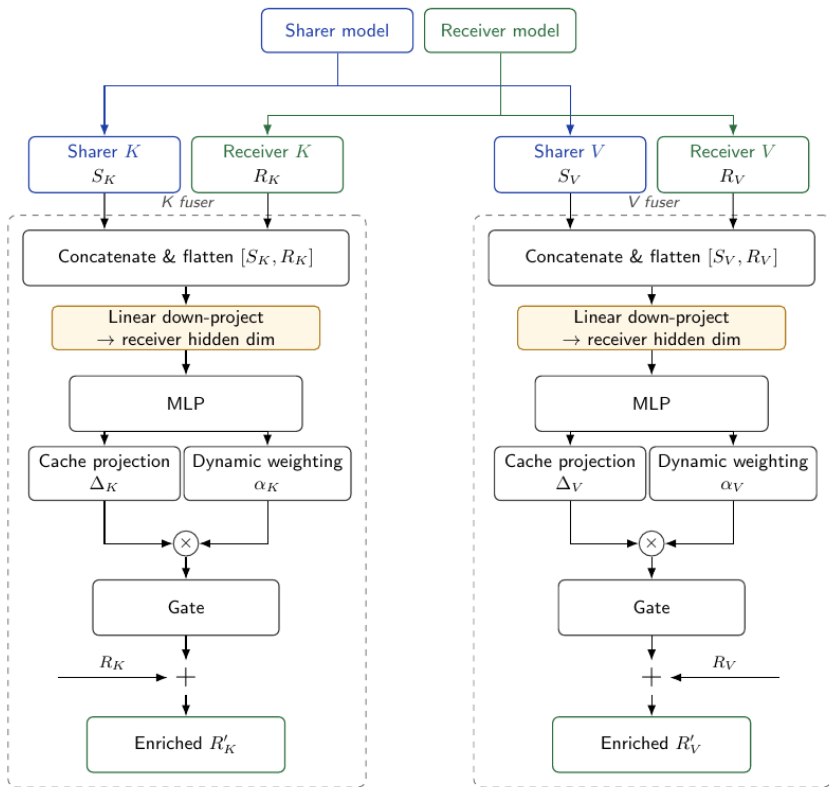
## C2C의 핵심 수식

$$C_F^n = C_R^n(X) + F_n\left(C_R^n(X), C_S^{G(n)}(X)\right)$$

$$y_{i+1} = P(y_i \mid C_F(X) \oplus C(Y_{0:i}))$$



C2C는 receiver가 이미 같은 input X를 prefill해서 만든 cache를 가지고 있고, 여기에 sharer cache를 residual하게 섞는 구조다.



# C2C에서 fusion을 쓰는 이유

## 왜 receiver KV를 같이 보나?

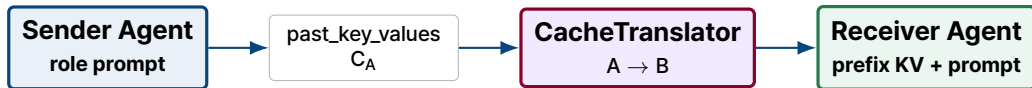
- ▶ Sharer와 Receiver가 같은 context X를 인코딩한다.
- ▶ Receiver의 기존 이해를 보존해야 한다.
- ▶ Gate가 도움이 되는 layer만 선택한다.

## 우리 설정과 다른 점

- ▶ agent마다 role prompt와 objective가 다르다.
- ▶ Receiver KV를 만들려면 receiver prefill을 먼저 해야 한다.
- ▶ 우리는 receiver prefill 전에 prefix memory를 주입하고 싶다.

따라서 HeteroMAS는 C2C-style fusion이 아니라 **sender-only KV translation + prefix prepend**를 기본 구조로 둔다.

## HeteroMAS: KV prefix translation



### 실행 순서

$A \text{ forward} \rightarrow C_A \rightarrow \hat{C}_{A \rightarrow B} \rightarrow B \text{ forward}(\text{past\_key\_values} = \hat{C}_{A \rightarrow B})$

중요: receiver KV를 먼저 만들고 섞는 것이 아니라, receiver가 자기 prompt를 처리하기 전에 translated KV가 prefix로 들어간다.

# 왜 receiver KV를 안 보나?

## Receiver KV를 보면

- ▶ Receiver prompt prefix이 먼저 필요하다.
- ▶ 이후 cache editing/fusion 문제가 된다.
- ▶ text 없이도 두 모델의 cache를 token-level로 맞춰야 한다.

## Sender-only translation이면

- ▶ sender의 latent memory만 receiver shape으로 바꾸면 된다.
- ▶ token length alignment가 필요 없다.
- ▶ receiver는 prefix를 보고 자기 role reasoning을 시작한다.



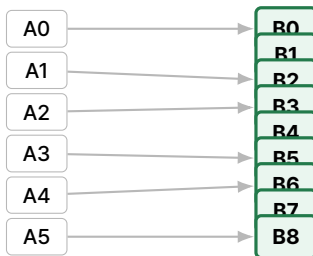
# Layer alignment: depth-normalized mapping

## Receiver-driven mapping

Receiver layer  $r \in \{0, \dots, L_B - 1\}$ 에 대해 sender layer를 선택한다.

$$s(r) = \text{round}\left(r \cdot \frac{L_A - 1}{L_B - 1}\right)$$

$$\ell_A = s(r), \quad \ell_B = r$$



one-to-many 또는 many-to-one이 생길 수 있지만, layer depth semantics를 보존한다는 점에서 all-layer concat/split보다 안정적이다.



# CacheTranslator: token-wise MLP projection

## Sender KV

$$\mathbf{K}_A^{s(r)}, \mathbf{V}_A^{s(r)} \in \mathbb{R}^{B \times H_A \times T_A \times d_A}$$

## Head와 head dimension만 flatten, token 축은 유지

$$\mathbf{X}_A^r = \text{Flatten}([\mathbf{K}_A^{s(r)}, \mathbf{V}_A^{s(r)}]) \in \mathbb{R}^{B \times T_A \times 2H_A d_A}$$

$$\mathbf{Y}_A^r = \mathbf{W}_{2\sigma}(\mathbf{W}_1 \mathbf{X}_A^r + \mathbf{e}_r) \in \mathbb{R}^{B \times T_A \times 2H_B d_B}$$

## Receiver shape으로 복원

$$(\hat{\mathbf{K}}_{A \rightarrow B}^r, \hat{\mathbf{V}}_{A \rightarrow B}^r) = \text{Reshape}_B(\mathbf{Y}_A^r) \in \mathbb{R}^{B \times H_B \times T_A \times d_B}$$

## Receiver attention: prepend 방식

### Translated prefix KV

$$\hat{\mathbf{C}}_{A \rightarrow B} = \{(\hat{\mathbf{K}}_{A \rightarrow B}^r, \hat{\mathbf{V}}_{A \rightarrow B}^r)\}_{r=0}^{L_B-1}$$

### Receiver가 자기 prompt를 처리할 때

$$\begin{aligned} \mathbf{K}_{\text{total}}^r &= [\hat{\mathbf{K}}_{A \rightarrow B}^r; \mathbf{K}_B^r], & \mathbf{V}_{\text{total}}^r &= [\hat{\mathbf{V}}_{A \rightarrow B}^r; \mathbf{V}_B^r] \\ \text{Attn}(\mathbf{Q}_B^r, \mathbf{K}_{\text{total}}^r, \mathbf{V}_{\text{total}}^r) &= \text{softmax}\left(\frac{\mathbf{Q}_B^r (\mathbf{K}_{\text{total}}^r)^\top}{\sqrt{d_B}}\right) \mathbf{V}_{\text{total}}^r \end{aligned}$$

토큰 길이  $T_A$ 를  $T_B$ 에 맞추는 필요가 없다. Prefix sequence가 앞에 추가될 뿐이고, receiver의 실제 prompt token은 그대로 유지된다.

# Multi-agent 구조에서의 KV 전달

## Sequential

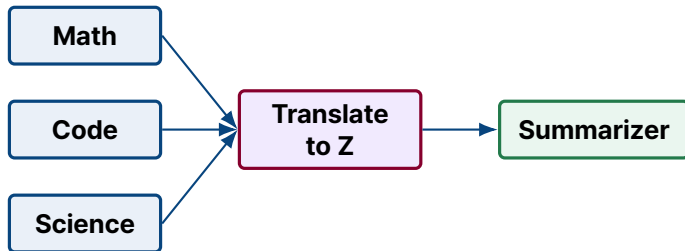
Planner  $\rightarrow \hat{C}_{P \rightarrow R} \rightarrow$  Refiner  $\rightarrow \hat{C}_{R \rightarrow S} \rightarrow$  Solver

Recursive round가 있으면 solver KV를 다음 round planner prefix로 번역한다.

## Hierarchical

$\hat{C}_{M \rightarrow Z} \oplus \hat{C}_{C \rightarrow Z} \oplus \hat{C}_{S \rightarrow Z} \rightarrow$  Summarizer

각 expert KV를 summarizer-shaped KV로 번역하고 sequence dimension으로 concat한다.



## 구조 비교

| Method       | 전달 단위                          | Heterogeneous | 학습             | 우리와의 차이                                          |
|--------------|--------------------------------|---------------|----------------|--------------------------------------------------|
| LatentMAS    | last hidden + KV memory        | 약함            | training-free  | raw KV transfer 중심, shape mismatch가 main 이 아님    |
| RecursiveMAS | hidden state / recursive link  | 가능            | inner/outer 학습 | KV attention memory가 아니라 hidden 전달               |
| C2C          | sender KV + receiver KV fusion | pairwise 가능   | fuser 학습       | 같은 input cache를 fusion 하는 구조                     |
| HeteroMAS    | translated KV prefix           | 목표 설정         | translator 학습  | role-specialized agent 사이 sender-only KV prepend |

HeteroMAS의 novelty는 “latent communication” 자체가 아니라, heterogeneous role agents 사이에서 KV cache 를 receiver-compatible prefix memory로 번역하는 것이다.

## 정리

1. LatentMAS는 text decoding을 줄이지만 homogeneous KV working memory 가정이 강하다.
2. RecursiveMAS는 heterogeneous MAS 구조를 갖지만 hidden-state 전달과 role-level supervision 부담이 있다.
3. C2C는 KV의 semantic transfer 가능성을 보였지만 receiver KV와의 fusion이 핵심이다.
4. HeteroMAS는 RecursiveMAS의 비교 구조를 유지하되, cross-agent 전달을 hidden state가 아니라 KV prefix translation으로 바꾼다.

